

Lecture 8: Damped Newton; Introduction to Conditioning

Math 6610

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Today's Goals

- Identify why Newton's method diverges from a poor initial guess
- Damping
- Algorithm in MATLAB
- Introduce conditioning

Newton Can Fail: a One-Line Example

Solve $f(x) = \arctan x = 0$ (answer: $x^* = 0$). Newton from $x^0 = 2$:

k	0	1	2	3	4	5
x^k	2	-3.54	13.95	-279	$1.2 \cdot 10^5$	$-2.3 \cdot 10^{10}$

From $x^0 = 1$ the same code converges in 4 steps. The code is fine.

Remark (*What went wrong*). Newton solves the tangent line, not f . At $x^0 = 2$ the tangent is nearly flat, so it crosses the axis far away, where it says nothing about f . Lecture 6: $h = \alpha\gamma \geq \frac{1}{2}$.

The Fix: Shorten the Step

$\Delta x^k = -Df(x^k)^{-1}f(x^k)$ points the right way. It is only too long. Take part of it:

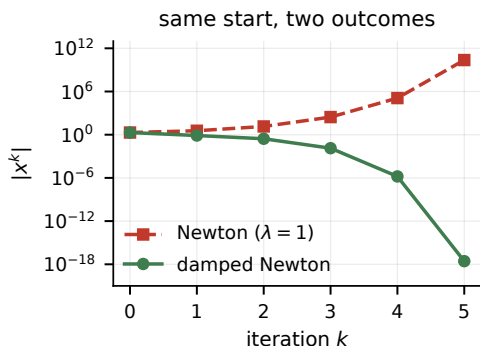
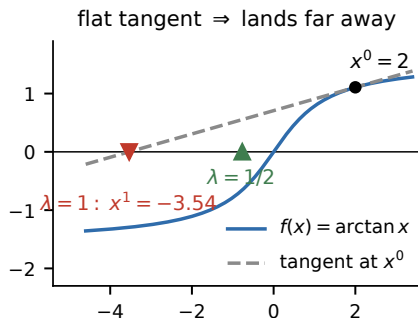
$$x^{k+1} = x^k + \lambda_k \Delta x^k, \quad 0 < \lambda_k \leq 1$$

- $\lambda_k = 1$: plain Newton. What we want near x^* .
- $\lambda_k < 1$: shorter step, new Jacobian, try again.

Remark (*Two rules*).

- 1 Damping must turn itself off near x^* , or quadratic convergence is lost.
- 2 We need a test for a good step, unchanged by rescaling the equations.

What Damping Does



- **Left:** slope $\frac{1}{5}$ at $x^0 = 2$, so the tangent hits the axis at -3.54 . Half the step lands at -0.77 , still near the root.
- **Right:** same start, same direction, two outcomes. One halving at $k = 0$ is the whole difference.

Recap: the Progress Test (Lecture 7)

Measure the new residual through $Df(x^k)^{-1}$:

$$\|Df(x^k)^{-1}f(x^{k+1})\| \leq \theta \underbrace{\|Df(x^k)^{-1}f(x^k)\|}_{=\|\Delta x^k\|}, \quad \theta < 1. \quad (1)$$

- Left side = $\|\overline{\Delta x}^{k+1}\|$, the *simplified Newton correction*: solve $Df(x^k) \overline{\Delta x}^{k+1} = -f(x^{k+1})$.
- Same matrix, so reuse the LU factors: $O(n^2)$, not $O(n^3)$.
- Unchanged when $f \mapsto Mf$; $\|f(x^k)\|$ is not.

Remark. A small enough λ_k always passes (1) (box later). So: shrink until it passes.

The Damped Newton Algorithm

Given x^k and a starting λ ($\lambda = 1$ at $k = 0$):

- ❶ Solve $Df(x^k) \Delta x^k = -f(x^k)$ (factor once; keep L and U).
- ❷ For $\lambda = \lambda, \frac{\lambda}{2}, \frac{\lambda}{4}, \dots, \lambda_{\min}$:
 - ▶ trial point $x^{k+1}(\lambda) = x^k + \lambda \Delta x^k$;
 - ▶ solve $Df(x^k) \overline{\Delta x}^{k+1}(\lambda) = -f(x^{k+1}(\lambda))$ (reuse LU);
 - ▶ accept if

$$\|\overline{\Delta x}^{k+1}(\lambda)\| \leq \left(1 - \frac{\lambda}{2}\right) \|\Delta x^k\|. \quad (2)$$

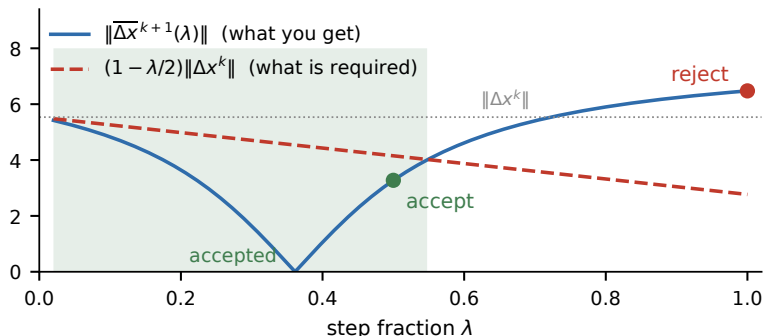
- ❸ $\lambda_k =$ the accepted λ , $x^{k+1} = x^k + \lambda_k \Delta x^k$.
- ❹ Start the next step at $\lambda = \min\{1, 2\lambda_k\}$.

Remark. $\theta = 1 - \frac{\lambda}{2}$ changes with λ : a full step must halve the correction, a short step needs less.

The Line Search, Seen

Test (2) at $x^0 = 2$, against the trial λ :

line search at $x^0 = 2$: halve until blue is below red



- The full step ($\lambda = 1$) leaves a bigger correction than we started with.
- Halving from 1 first passes at $\lambda = \frac{1}{2}$.

The Same Bad Start, Damped

$f = \arctan$, $x^0 = 2$, where plain Newton blew up:

k	x^k	λ tried	$\ \Delta x^k\ $
0	2	1: $6.48 \not\leq 2.77$ reject; $\frac{1}{2}$: $3.27 \leq 4.15$ accept	5.5
1	-0.768	1: accept	1.0
2	0.273	1: accept	0.29
3	-0.0134	1: accept	$1.3 \cdot 10^{-2}$
4	$1.6 \cdot 10^{-6}$	1: accept	$1.6 \cdot 10^{-6}$
5	$-2.7 \cdot 10^{-18}$		

Remark. One halving was enough. After that $\lambda_k = 1$ and the errors square: $10^{-2} \rightarrow 10^{-6} \rightarrow 10^{-18}$. At $k = 0$ the full step made things worse ($6.48 > 5.54$); the test caught it.

MATLAB codes

```
[L,U,P] = lu(Df(x)); % ONE factorization per Newton step
dx = U \ (L \ (P*(-f(x)))); % Newton correction
if norm(dx) <= tol, return, end % stopping rule (Lecture 7)

lam = min(1, 2*lam); % try to grow back to a full step
while lam >= lmin
    xt = x + lam*dx; % trial point
    dbar = U \ (L \ (P*(-f(xt)))); % simplified correction: reuse LU
    if norm(dbar) <= (1 - lam/2)*norm(dx)
        break % test passed
    end
    lam = lam/2; % rejected: halve and retry
end
if lam < lmin, error('check Df or the starting point'); end
x = x + lam*dx;
```

- The one `lu` is the only $O(n^3)$ work; both solves are $O(n^2)$.
- The inner `while` loop is all of damping.

MATLAB: Two Test Cases

```
f = @(x) atan(x);    Df = @(x) 1/(1+x^2);
[~, hd, ld] = dampnewton(f, Df, 2);
% ld = 0.5  1  1  1  1      <- one halving, then full steps
% hd = 2  -0.77  0.27  -1.3e-2  1.6e-6  -2.7e-18

f2 = @(x) [x(1)^2 + x(2)^2 - 1; x(2) - x(1)^2]; % circle & parabola
Df2 = @(x) [2*x(1), 2*x(2); -2*x(1), 1];
[x2, h2, l2] = dampnewton(f2, Df2, [0.02; 4]);
% l2(1:5) = 0.031  0.063  0.125  0.25  0.5
% largest |x| visited: 4.0      (plain Newton swings out to 47)
```

Remark (*What test 2 shows*). Plain Newton converges here too, in the same number of steps, but first flies out to $|x_1| \approx 47$. Damping saved no steps here. It kept the iterates near the solution.

Practical Notes

- **Cost.** One LU per step ($\frac{2}{3}n^3$); each trial λ costs one f and one back-substitution ($O(n^2)$). Trials are cheap.
- $\lambda_{\min}$. Stop the inner loop at 10^{-4} . Reaching it means the direction is bad: Df nearly singular, a poor start, or a wrong Jacobian. Report failure.
- **Check the Jacobian.** One wrong entry looks exactly like “needs damping”. Test Df against finite differences.
- **Do not test $\|f\|$.** Scaling one equation by 10^6 changes $\|f\|$, not the iterates.
- **Free near the end.** Doubling returns to $\lambda_k = 1$ as soon as it can.

Theory: Why Shrinking Always Works

Proposition (the Newton direction goes downhill)

Let $f(x^k) \neq 0$, $\|v\| := \sqrt{v^T v}$, and $\phi(x) := \frac{1}{2} \|Df(x^k)^{-1}f(x)\|^2$. Then

$$\left. \frac{d}{d\lambda} \phi(x^k + \lambda \Delta x^k) \right|_{\lambda=0} = -\|\Delta x^k\|^2 < 0.$$

Proof

Put $g(x) := Df(x^k)^{-1}f(x)$, so $\phi = \frac{1}{2}g^T g$ and $D_x \phi(x) = g(x)^T Df(x^k)^{-1} Df(x)$. At $x = x^k$, $Df(x^k)^{-1} Df(x^k) = I$ and $g(x^k) = -\Delta x^k$, so $D_x \phi(x^k) = -(\Delta x^k)^T$. Multiply by Δx^k .

So ϕ decreases for small $\lambda > 0$: (1) holds once λ_k is small enough, with $\phi(x^{k+1}) \leq \theta^2 \phi(x^k)$. *The inner loop always stops.*

Theory: Where $\theta = 1 - \lambda/2$ Comes From

First-order model

With g as above, $Dg(x^k) = I$, so

$$g(x^k + \lambda \Delta x^k) = g(x^k) + \lambda \underbrace{Dg(x^k) \Delta x^k}_{= \Delta x^k = -g(x^k)} + O(\lambda^2) = (1 - \lambda) g(x^k) + O(\lambda^2).$$

At best, a step of length λ shrinks the correction by $1 - \lambda$; a full step removes it, to first order.

The test

Ask for half of that: $\theta = 1 - \frac{\lambda}{2}$. Some λ always passes, and steps spoiled by the nonlinearity are rejected. (Armijo condition, from optimization.)

New Topic: How Much Accuracy Is Possible?

Error in a computed answer has two sources:

$$\text{error} = \underbrace{\text{sensitivity of the } \textit{problem}}_{\text{conditioning} - \text{today}} \times \underbrace{\text{quality of the } \textit{algorithm}}_{\text{stability} - \text{later}}.$$

Remark (*Why this comes first*). If the problem is sensitive, no algorithm helps: the data itself does not determine the answer that accurately.

A problem is a map from input to output,

$$F : X \rightarrow Y, \quad x \in X \text{ (input)}, \quad y = F(x) \in Y \text{ (output)}.$$

Example (*Our running case*). Solve $Ay = b$: $x = (A, b)$ with $A \in \mathbb{C}^{n \times n}$ nonsingular and $b \in \mathbb{C}^n$; $Y = \mathbb{C}^n$; $F(A, b) := A^{-1}b$.

The Condition Number

Change the input, watch the output:

$$x \rightarrow x + \delta x, \quad \delta F(x) := F(x + \delta x) - F(x).$$

Definition (condition number)

$\kappa = \kappa(x)$ is the smallest number with

$$\frac{\|\delta F(x)\|}{\|F(x)\|} \leq \kappa(x) \frac{\|\delta x\|}{\|x\|} \quad \text{for all small } \delta x.$$

Relative error in, relative error out.

Remark. κ belongs to the problem, not the algorithm, and depends on x : the same formula can be fine at one x and useless at another (next lecture: $x_1 - x_2$).

Reading κ : How Many Digits Survive

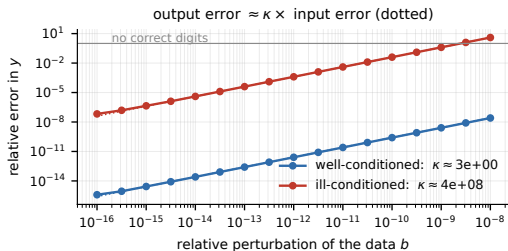
Data known to ≈ 16 digits ($\epsilon_{\text{mach}} \approx 10^{-16}$):

$\kappa(x)$	1	10^2	10^8	10^{14}	10^{16}
digits you can trust	16	14	8	2	0

$$\text{lost digits} \approx \log_{10} \kappa(x).$$

Remark (*How to use this*). Estimate κ before blaming the solver. If $\kappa = 10^{14}$ and you need 6 digits, change the problem, not the algorithm.

Conditioning, Measured



```
A = [1 1; 1 1+1e-8];  
y = [1; 1]; b = A*y;  
d = 1e-14*norm(b)*[1;-1]/sqrt(2);  
norm(A\b(b+d) - y)/norm(y)  
% ans = 4.0e-06  
cond(A)  
% ans = 4.0e+08
```

- Change b in the last digit (10^{-14}): the answer moves by $4 \cdot 10^{-6}$. Eight digits gone, and \ did nothing wrong.
- Dotted lines: $\kappa \cdot$ (input error). The bound is actually reached.

Looking Ahead

- **Next lecture:** Conditioning; Floating-Point Numbers